

DSAA 4011 · LAB 5 · SATURDAY 10 OCTOBER 2026

Learning What We Never See

Expectation–Maximization at the Frontier

Lecture 5's **expectation–maximization** (EM): **guess** what is hidden, **count**, repeat. Today: five places where this simple loop runs modern AI.

Yuwen Huang

Advanced Machine Learning and Deep Learning · HKUST (Guangzhou)

Builds on Lecture 5: Learning what we never observe

Outline

- ▶ **Learning what we never see (Lecture 5)**

Troubles 1–2: cleaning and searching data

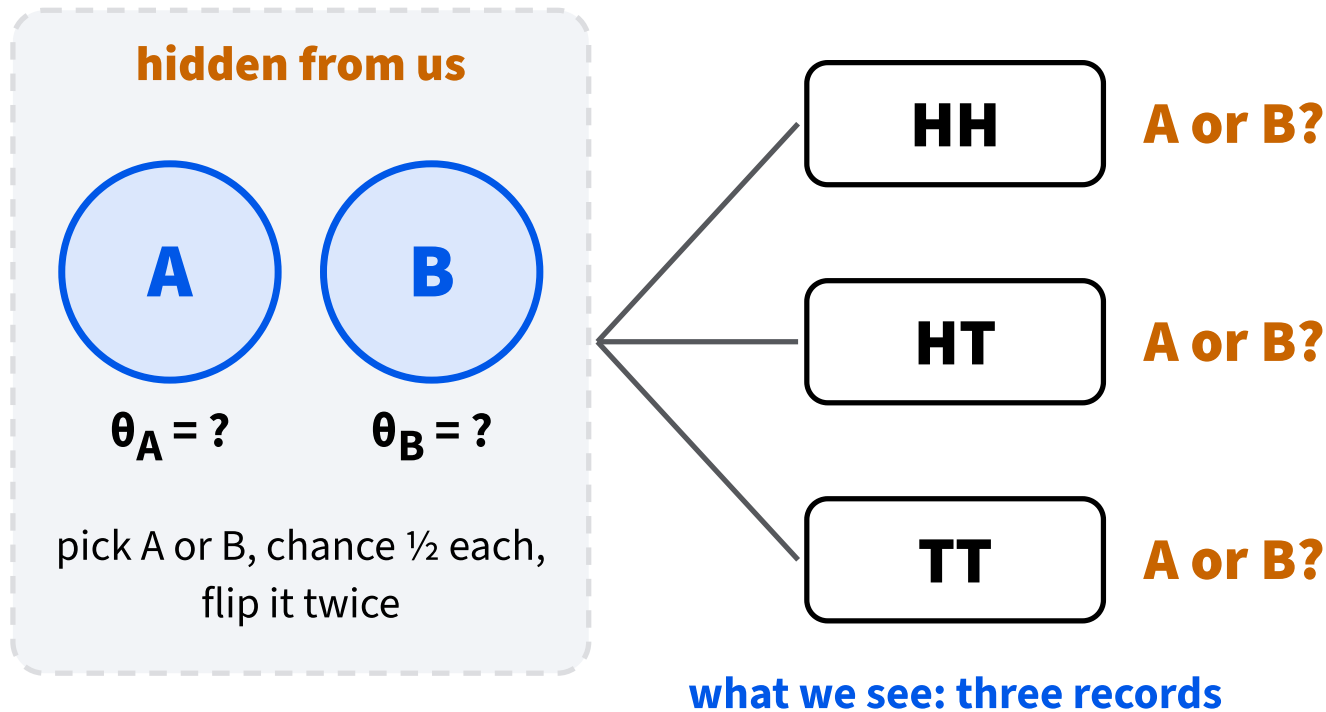
Troubles 3–5: inside image and language models

Conclusion and your turn

One loop, five troubles, five small fixes.

Learning what we never see

Two coins, one hidden: which coin made each record?



Look: we see the flips, **never the coin**.

Q: Can we learn θ_A and θ_B anyway?

A: Yes: guess the coin **softly**, count heads, repeat.

Learning what we never see: in three sentences

BACKGROUND Data hide part of the story: which coin, which label, which reasoning.

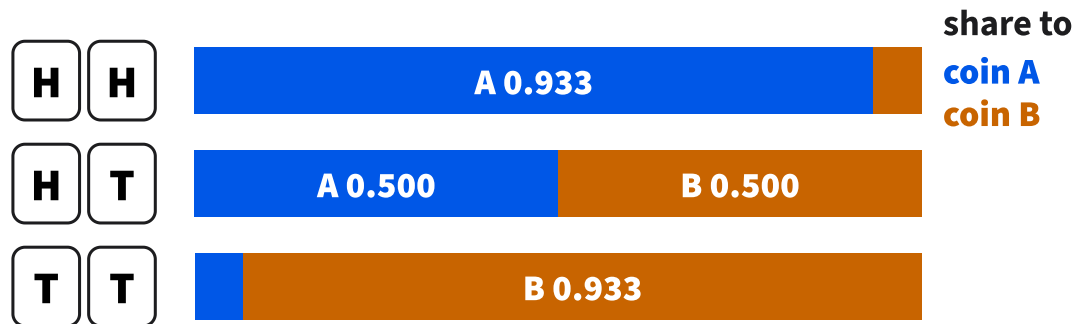
KEY IDEA **Guess** the hidden part softly (**E-step**), then **count** (**M-step**). Repeat.

USED FOR Every model with a hidden part [Dempster, Laird and Rubin, 1977].

Learning what we never see

Guess **softly**, **count**, repeat.

round 15 · M-step: count



heads and flips, credited by the soft labels:

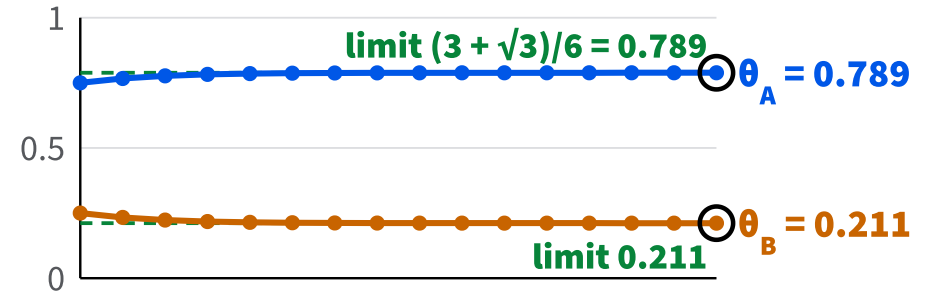
heads to A: $2 \cdot 0.933 + 1 \cdot 0.500 + 0 \cdot 0.067 = 2.366$

flips to A: $2 \cdot (0.933 + 0.500 + 0.067) = 3.000$

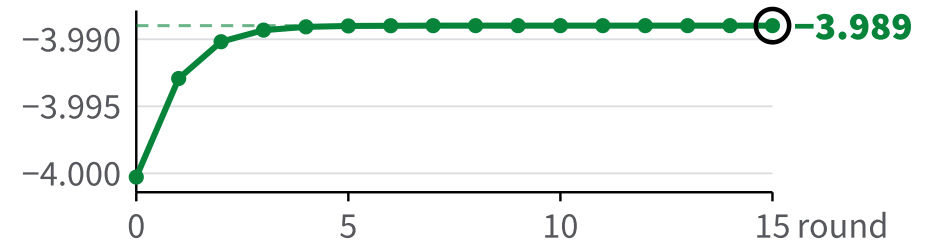
$\theta_A = 2.366 \div 3.000 = 0.789$

$\theta_B = 0.634 \div 3.000 = 0.211$

heads chances



log-likelihood $\ln p(\text{HH}, \text{HT}, \text{TT})$: never goes down



Watch: HH looks like coin A: **soft label** 0.9 (p. 32). Round by round θ_A climbs to **0.789**, and the fit (likelihood) **never drops**.

Q: Why not pick one coin per record? **A:** HT is 50 : 50; a hard pick is far too sure.

Try: another start.

Main Question

**EM only guesses and counts.
Can it judge a crowd, search a library, read images, cut
words, and teach reasoning?**

Yes: one small change each.

Learning what we never see

Five words for today.

Word	Plain meaning	Lecture 5 name
record x	what we see	observed data
hidden coin z	what we never see	hidden variable
soft label	how much we believe each z	posterior, E-step
count	heads weighted by soft labels	M-step
likelihood	chance of all records; never drops	Theorem 4

Learning what we never see

Five troubles at the frontier, five fixes.

	Trouble	Fix	Used for
1	people disagree on labels	???	clean training labels
2	too much data	???	search for chatbots
3	the hidden part is many numbers	???	image models
4	nobody marks where to cut words	???	tokenizers
5	the reasoning is never written	???	models that reason

Read: the fixes appear one by one.

Outline

Learning what we never see (Lecture 5)

- ▶ **Troubles 1–2: cleaning and searching data**

Troubles 3–5: inside image and language models

Conclusion and your turn

First: clean the labels, then search the data.

Trouble 1 in three sentences

- BACKGROUND** An AI learns from photos tagged “cat” or “dog” by people. Wrong tags teach it wrong.
- KEY IDEA** Find the **true tags** and **whom to trust** together: each one helps find the other.
- USED FOR** Cleaning crowd-sourced labels before training.

Trouble 1: people disagree

Trouble: three people tag the same photos, and **disagree**.

	photo 1	photo 2
person 1	cat	dog
person 2	cat	dog
person 3	dog	dog
true label	?	?

1. Photo 1: two say **cat**, one says **dog**.

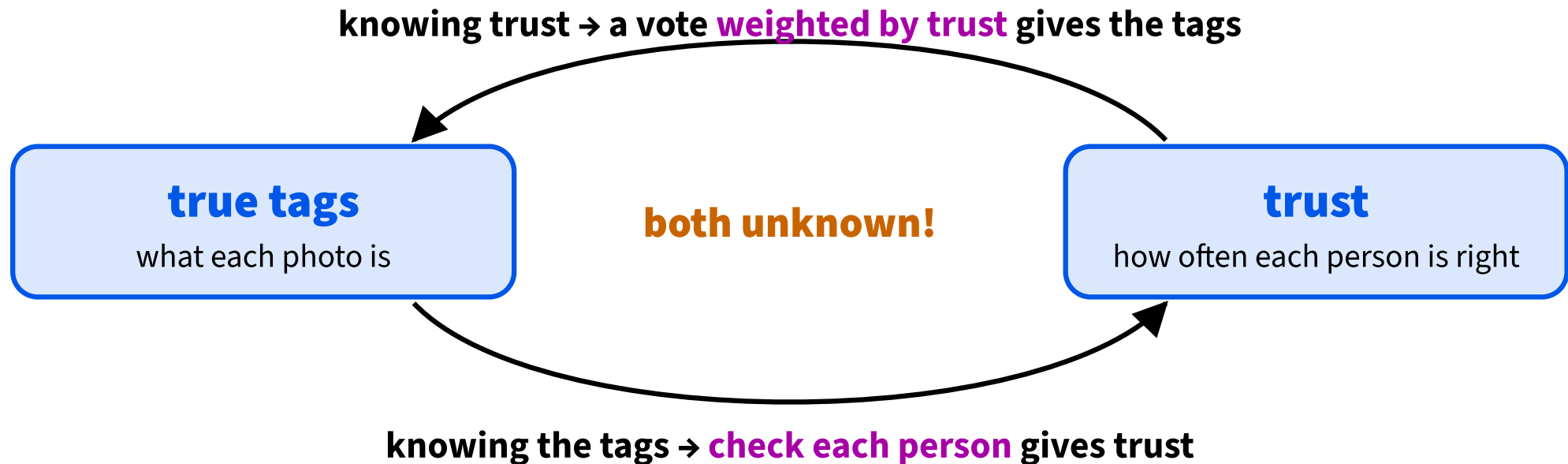
2. A vote says cat, but it trusts **everyone** **equally**.

Q: What if person 3 is careless?

A: Then their vote should count less. But who is careless?

Trouble 1: people disagree

Trouble: a chicken-and-egg problem.



Read: each unknown would give the other, but we have **neither**. Same as Lecture 5: the true tag is the hidden coin; trust is θ .

Q: Where do we start? A: Guess trust, then go round the loop: that is EM.

Trouble 1: people disagree

Fix: go round the loop. One round by hand:

START Trust everyone the same: **0.8**.

① **TAGS** Photo 1: cat, cat, dog → **80% cat**. Two votes beat one (**p. 34**).

② **TRUST** Person 3 went against the others on 2 of 6 photos → trust **0.8** → **0.72**.

REPEAT Person 3's vote now counts less, so the tags get surer, and person 3's trust settles at **0.67**.

Read: nobody told EM who is careless; it found out from the **disagreements**.

Trouble 1: people disagree

Result: after a few rounds, EM knows **whom to trust**.

round 8 • ② score each person

	person 1	person 2	person 3	what is the photo?
photo 1	cat	cat	dog	cat 1.000
photo 2	dog	dog	dog	dog 1.000
photo 3	cat	cat	cat	cat 1.000
photo 4	dog	dog	cat	dog 1.000
photo 5	cat	cat	cat	cat 1.000
photo 6	dog	dog	dog	dog 1.000

person 3 matches the guesses on photos 1 ... 6:

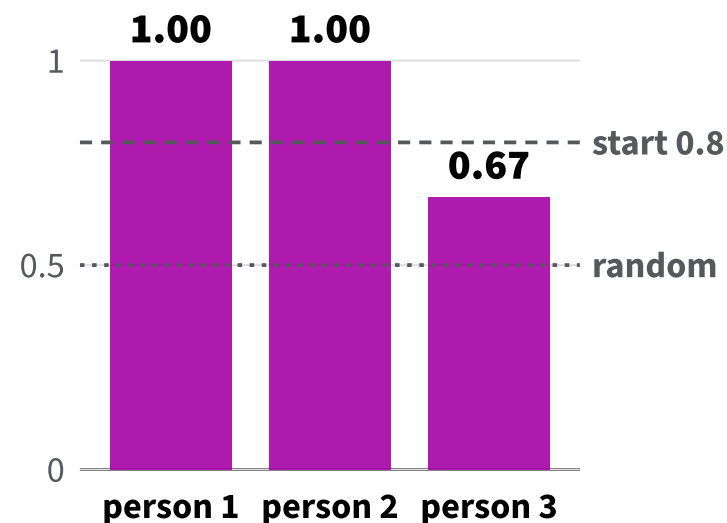
$$(0.000 + 1.000 + 1.000 + 0.000 + 1.000 + 1.000) \div 6 = 0.667$$

Watch: each round does ① then ②. The photos become sure (blue), person 3's trust falls to **0.67**, persons 1 and 2 rise to 1.

Q: Why does person 3 lose trust? A: They disagree with two people who agree with each other.

Try: person 3 guesses at random: trust 0.5, their tags stop counting. This is **Dawid-Skene EM** [Dawid and Skene, 1979].

trust: how often each person is right



persons 1 and 2 always agree:
EM ends up trusting them fully

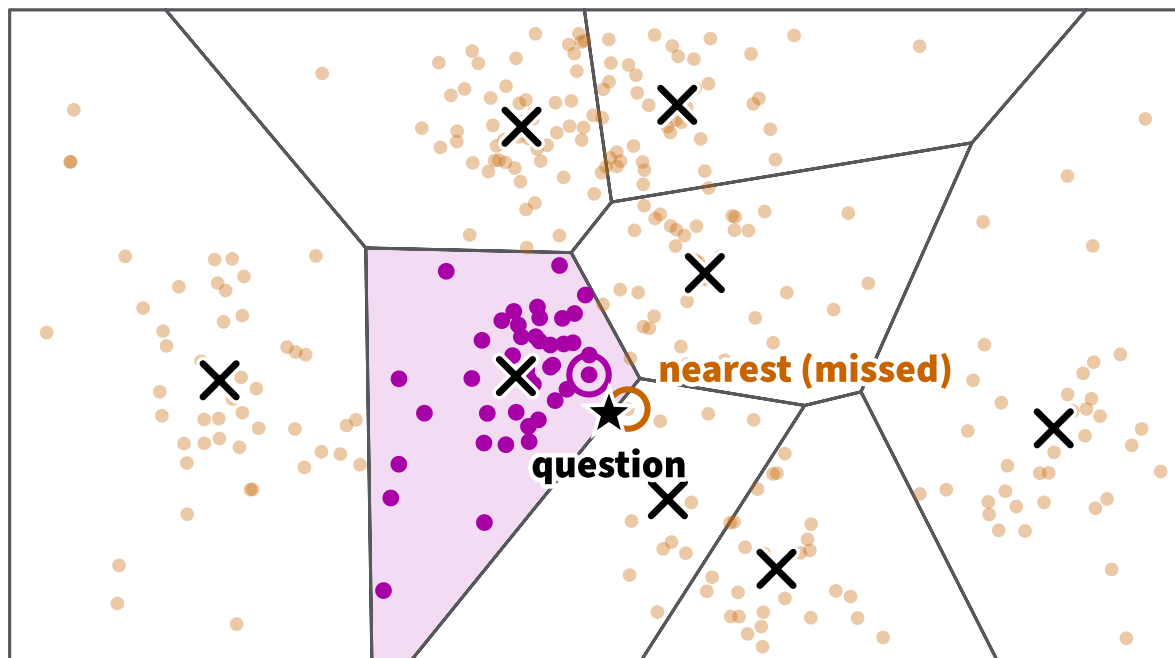
Trouble 2 in three sentences

- BACKGROUND** A chatbot stores millions of texts as **points**: similar meaning, close points.
- KEY IDEA** Like a library: shelve points by topic (**k-means**, EM with **hard labels**); search **one shelf**.
- USED FOR** Vector search behind chatbots [Jégou, Douze and Schmid, 2011; Lewis et al., 2020].

Trouble 2: too much data

Trouble: millions of texts. **Fix:** search one cell.

texts as vectors (orange) · centres (black x)



search one cell (8 cells)

every text: 300

one cell: $8 + 41 = 49$

nearest text: missed x

magenta ring: best in the cell, not the nearest

**over 50 questions: 4% of nearest texts missed,
about 50 distance computations each, not 300**

Watch: make each soft label hard: every point joins its nearest centre (Theorem 5). EM becomes **k-means**.

Q: Why not check every text? A: 1,000 cells: 500 times fewer distances.

Try: more cells: faster, but more **misses**. This is **k-means search** [Lloyd, 1982].

Outline

Learning what we never see (Lecture 5)

Troubles 1–2: cleaning and searching data

► **Troubles 3–5: inside image and language models**

Conclusion and your turn

Next: inside image and language models.

Trouble 3 in three sentences

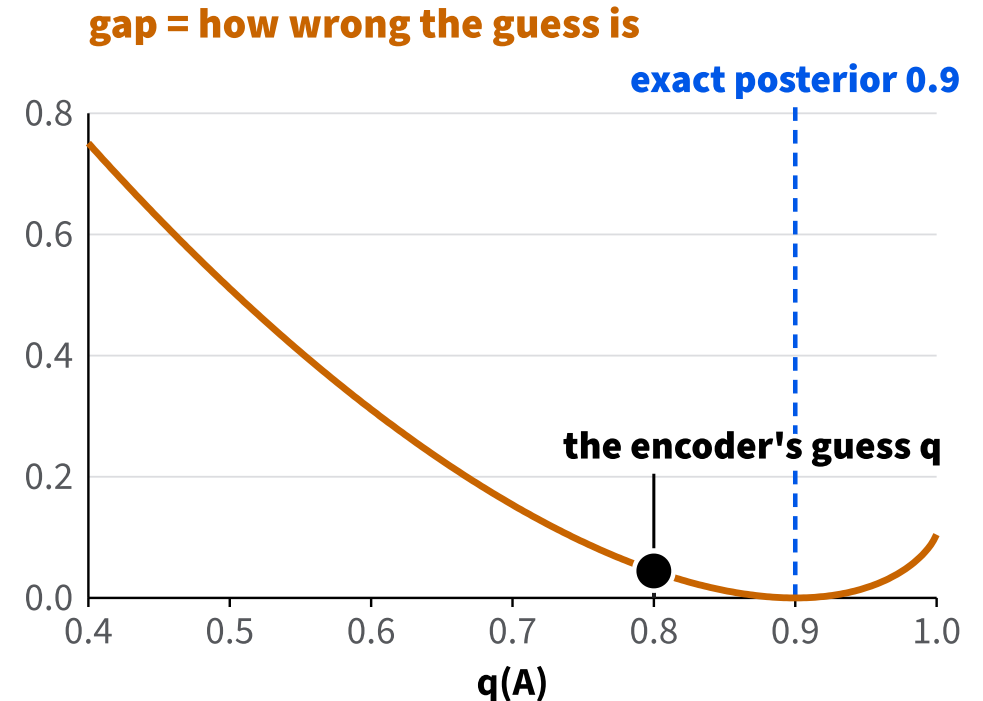
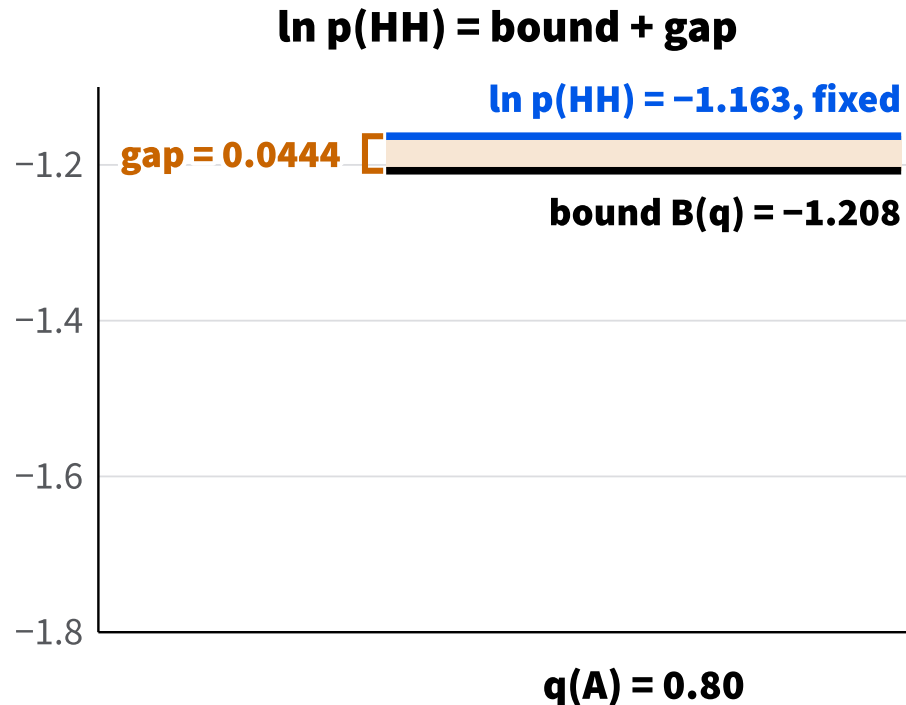
BACKGROUND An image model describes each image by a hidden **code**: many numbers.

KEY IDEA Too many codes to check; train a **fast guesser**, the **encoder**.

USED FOR Image models that compress, generate and edit.

Trouble 3: the code is many numbers

Trouble: endless codes. **Fix:** a fast guesser, the **encoder**.



Watch: a wrong guess leaves a **gap** under $\ln p(x)$; at the exact 0.9 the gap closes (p. 35).

Q: Why not compute it exactly? A: For images there is no formula.

Try: move the guess to 0.9. This is the **variational autoencoder** [Kingma and Welling, 2014].

Trouble 4 in three sentences

BACKGROUND Language models read text in pieces: un|happi|ness. Nobody marks the **cuts**.

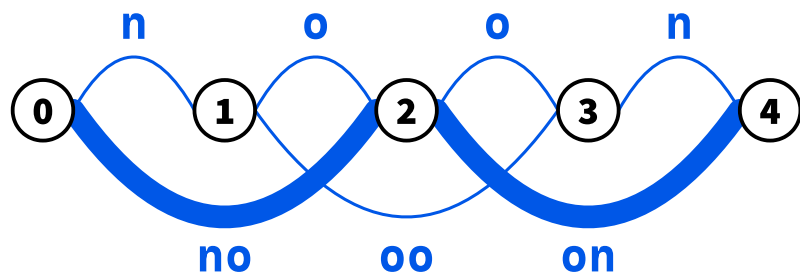
KEY IDEA The cut is the hidden coin; EM weighs **every cut** and counts pieces.

USED FOR Tokenizers of T5 and ALBERT [Kudo, 2018].

Trouble 4: where to cut words

Trouble: where to cut “noon”? **Fix:** weigh every cut.

every path 0 → 4 is a cut of “noon”



thicker arc = likelier piece

**the cut no|on becomes
almost certain (0.999)**

E-step: share of each cut
= its chance ÷ chance of “noon”

n|o|o|n | 0.000

n|o|on | 0.001

n|oo|n | 0.000

no|o|n | 0.001

no|on | 0.999

M-step: piece probability
= expected count ÷ 2.00 pieces

n | 0.001

o | 0.001

no | 0.499

on | 0.499

oo | 0.000

Watch: no|on starts ahead (0.61): fewer pieces. Counting boosts **no** and **on**, and no|on wins.

Q: Why not list every cut? A: 20 letters: up to 524,288 cuts. A running total adds them.

Try: favour “oo”: EM gets stuck. This is the **Unigram tokenizer** [Kudo, 2018].

Trouble 5 in three sentences

- BACKGROUND** Data give questions and answers, **never the reasoning**.
- KEY IDEA** Let the model try, **keep what works**, retrain.
- USED FOR** Teaching language models to reason [Zelikman et al., 2022].

Trouble 5: the hidden reasoning

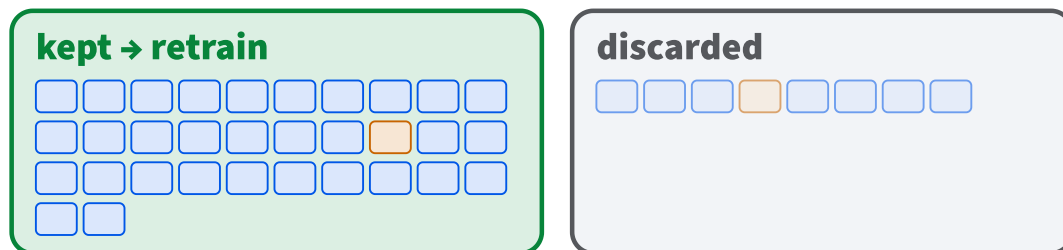
Trouble: reasoning is never written. **Fix:** keep what works.

the model answers 40 questions

☐ careful: right $\frac{3}{4}$ ☐ hasty: right $\frac{1}{4}$

kept 32: 31 careful, 1 hasty

retrain: new $s = 31/32 = 0.969$

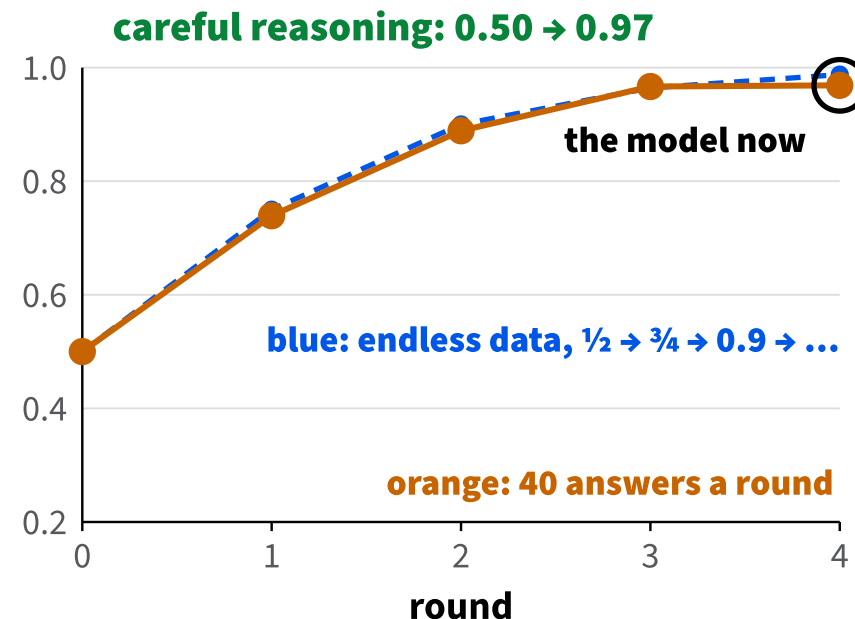


Watch: careful reasoning is right more often, so the kept traces are mostly careful: $\frac{1}{2} \rightarrow \frac{3}{4}$ (p. 36).

Q: Why not keep every trace? **A:** Then nothing changes.

Try: keep all. This is STaR, a **Monte Carlo EM** [Zelikman et al., 2022; Phan et al., 2023].

s = share of careful reasoning



Outline

Learning what we never see (Lecture 5)

Troubles 1–2: cleaning and searching data

Troubles 3–5: inside image and language models

► **Conclusion and your turn**

Finally: all five fixes, and your turn.

Conclusion

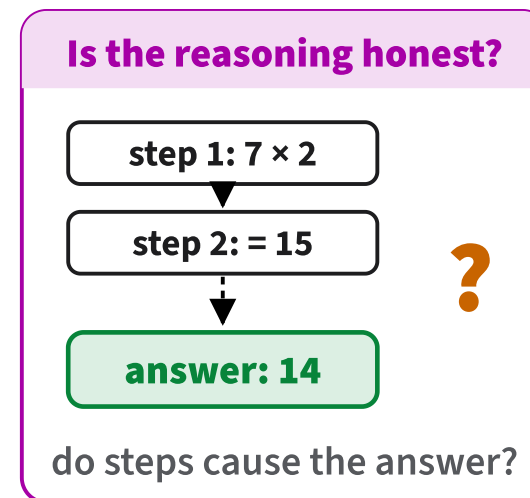
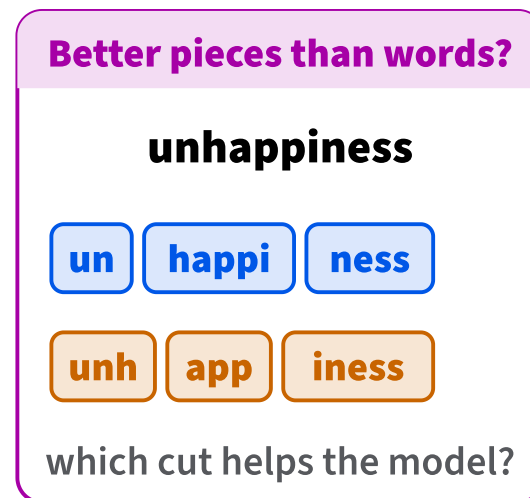
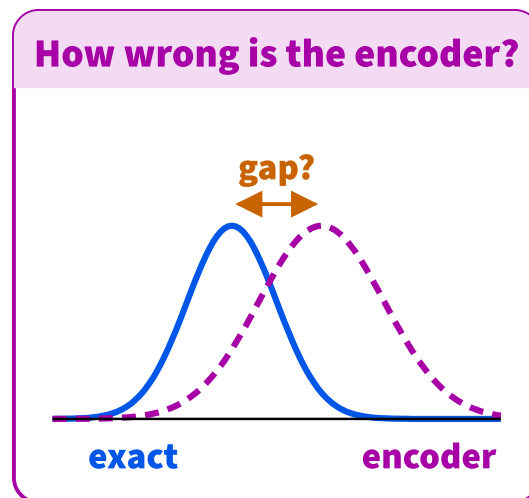
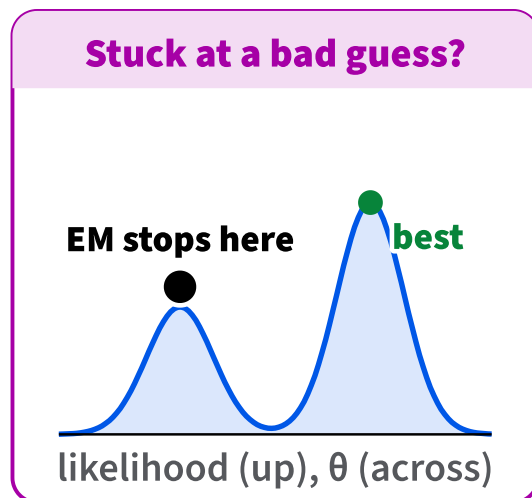
One loop, five small changes.

	Trouble	Fix	Used for
1	people disagree on labels	trust who agrees	clean training labels
2	too much data	search one cell	search for chatbots
3	the hidden part is many numbers	a fast guesser	image models
4	nobody marks where to cut words	weigh every cut	tokenizers
5	the reasoning is never written	keep what works	models that reason

Read: every fix keeps **guess**, then **count**.

Conclusion

- ▶ We can **learn what we never see**: guess, then count (expectation-maximization).
- ▶ Four **open problems**:



Project seeds: Jin et al. (2016), Cremer et al. (2018), Pagnoni et al. (2025), Turpin et al. (2023).

Your turn

Predict, then try.

In these slides

- ▶ another start (p. 5): still 0.789?
- ▶ a careless labeller (p. 14)
- ▶ more cells (p. 16)
- ▶ the encoder's guess (p. 19)
- ▶ “oo” first (p. 21); keep all (p. 23)

After class: lab5_explore.py

1. Two coins from $\theta = (0.6, 0.6)$: where to?
2. Eight photos: how much is person 3 trusted?
3. k-means: how fast, how many misses?
4. A tiny reasoner: keep right, or keep all?

Course site → Week 5 → Lab files. One or two lines each.

Tip: write your guess first; surprises teach.

Selected References I

Start here: Dempster, Laird and Rubin (1977), where EM got its name.



Cremer, C., Li, X., and Duvenaud, D. (2018).

Inference suboptimality in variational autoencoders.

In Proceedings of the 35th International Conference on Machine Learning, PMLR 80:1078–1086.



Dawid, A. P. and Skene, A. M. (1979).

Maximum likelihood estimation of observer error-rates using the EM algorithm.

Journal of the Royal Statistical Society, Series C (Applied Statistics), 28(1):20–28.



Dempster, A. P., Laird, N. M., and Rubin, D. B. (1977).

Maximum likelihood from incomplete data via the EM algorithm.

Journal of the Royal Statistical Society, Series B (Methodological), 39(1):1–38.



Jégou, H., Douze, M., and Schmid, C. (2011).

Product quantization for nearest neighbor search.

IEEE Transactions on Pattern Analysis and Machine Intelligence, 33(1):117–128.



Jin, C., Zhang, Y., Balakrishnan, S., Wainwright, M. J., and Jordan, M. I. (2016).

Local maxima in the likelihood of Gaussian mixture models: structural results and algorithmic consequences.

In Advances in Neural Information Processing Systems 29, pages 4116–4124.

Selected References II

Entry points: Kingma and Welling (2014); Kudo (2018).



Kingma, D. P. and Welling, M. (2014).

Auto-encoding variational Bayes.

In 2nd International Conference on Learning Representations; arXiv:1312.6114.



Kudo, T. (2018).

Subword regularization: improving neural network translation models with multiple subword candidates.

In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, pages 66–75.



Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020).

Retrieval-augmented generation for knowledge-intensive natural language processing tasks.

In Advances in Neural Information Processing Systems 33, pages 9459–9474.



Lloyd, S. P. (1982).

Least squares quantization in pulse-code modulation.

IEEE Transactions on Information Theory, 28(2):129–137.

Selected References III

A short read: Zelikman et al. (2022), a model teaching itself to reason.



Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., Zhou, C., Yu, L., Weston, J., Zettlemoyer, L., Ghosh, G., Lewis, M., Holtzman, A., and Iyer, S. (2025).

Byte latent transformer: patches scale better than tokens.

In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 9238–9258.



Phan, D., Hoffman, M. D., Dohan, D., Douglas, S., Le, T. A., Parisi, A., Sountsov, P., Sutton, C., Vikram, S., and Sauros, R. A. (2023).

Training chain-of-thought via latent-variable inference.

In Advances in Neural Information Processing Systems 36.



Turpin, M., Michael, J., Perez, E., and Bowman, S. R. (2023).

Language models don't always say what they think: unfaithful explanations in chain-of-thought prompting.

In Advances in Neural Information Processing Systems 36.



Zelikman, E., Wu, Y., Mu, J., and Goodman, N. D. (2022).

STaR: bootstrapping reasoning with reasoning.

In Advances in Neural Information Processing Systems 35, pages 15476–15488.

Thank you!

Next: Lecture 6, when even the guess must be approximated

EM in 11 lines of Python: see the backup slides.

Backup: one round of Lecture 5

One round from $\theta = (\frac{3}{4}, \frac{1}{4})$, coins picked $\frac{1}{2}$ each.

$$p(A \mid \text{HH}) = \frac{9/32}{9/32 + 1/32} = \frac{9}{9 + 1} = 0.9$$

$$p(A \mid \text{HT}) = \frac{3}{3 + 3} = 0.5, \quad p(A \mid \text{TT}) = \frac{1}{1 + 9} = 0.1$$

$$\text{heads for A} = 0.9 \cdot 2 + 0.5 \cdot 1 + 0.1 \cdot 0 = 2.3$$

$$\text{flips for A} = (0.9 + 0.5 + 0.1) \cdot 2 = 3$$

$$\theta_A = \frac{2.3}{3} = 0.767, \quad \theta_B = \frac{3 - 2.3}{6 - 3} = 0.233$$

Notice: heads $\div$ flips, each record weighted by its soft label.

E-step: $\frac{1}{2} \cdot \frac{3}{4} \cdot \frac{3}{4} = 9/32$ against $\frac{1}{2} \cdot \frac{1}{4} \cdot \frac{1}{4}$

the same rule

soft label $\times$ heads

soft label $\times$ flips

M-step: count $\div$ total

Backup: one round of Lecture 5

EM in 11 lines of Python.

```
records = ["HH", "HT", "TT"]
a, b = 0.75, 0.25 # starting guess: heads chances of coins A and B
for r in range(1, 31):
    heads = flips = 0.0
    for x in records:
        h, t = x.count("H"), x.count("T")
        pa, pb = a**h * (1-a)**t, b**h * (1-b)**t # chance of the record from each coin
        s = pa / (pa + pb) # E-step: soft label for coin A
        heads += s * h; flips += s * (h + t) # count with the soft label
    a, b = heads / flips, (3 - heads) / (6 - flips) # M-step: expected heads / expected flips
    if r in (1, 2, 30): print(r, round(a, 3), round(b, 3))
1 0.767 0.233
2 0.777 0.223
30 0.789 0.211
```

Try: start from $a = b = 0.5$. What happens?

Backup: Trouble 1

One round on the six photos: every accuracy starts at 0.8, cat and dog $\frac{1}{2}$ each.

$$p(\text{cat} \mid \text{cat}, \text{cat}, \text{dog}) = \frac{0.064}{0.064 + 0.016} = 0.8$$

$$p(\text{dog} \mid \text{dog}, \text{dog}, \text{dog}) = \frac{0.256}{0.256 + 0.004} = 0.9846$$

$$a_1 = a_2 = \frac{0.8 + 0.8 + 4 \cdot 0.9846}{6} = 0.923$$

$$a_3 = \frac{0.2 + 0.2 + 4 \cdot 0.9846}{6} = 0.723$$

Notice: accuracy is counted like heads $\div$ flips, with “right” as heads.

E-step, photos 1 and 4

E-step, photos 2, 3, 5, 6: all agree

expected right $\div$ photos

person 3 is outvoted on photos 1 and 4

Backup: Trouble 3

The gap for the guess 0.8 on HH (exact: 0.9).

Lemma 1: $\ln p(x) = \text{bound} + \text{gap}$; the gap is the Kullback–Leibler divergence.

$$\begin{aligned}\text{gap} &= 0.8 \cdot \ln \frac{0.8}{0.9} + 0.2 \cdot \ln \frac{0.2}{0.1} \\ &= 0.8 \cdot (-0.1178) + 0.2 \cdot 0.6931 \\ &= -0.0942 + 0.1386 = 0.0444\end{aligned}$$

guess against exact, for A and for B

the two logarithms

the bound is 0.0444 too low

Notice: at the exact guess 0.9 both logs are 0: no gap.

Backup: Trouble 5

The tiny reasoner: careful = coin A, hasty = coin B, “right” = heads.

Careful: chance $\pi = \frac{1}{2}$, right $\frac{3}{4}$. **Hasty:** right $\frac{1}{4}$.

$$p(\text{careful and right}) = \frac{1}{2} \cdot \frac{3}{4} = \frac{3}{8}, \quad p(\text{hasty and right}) = \frac{1}{2} \cdot \frac{1}{4} = \frac{1}{8}$$

$$\text{new } \pi = \frac{3/8}{3/8 + 1/8} = 0.75$$

$$\text{keep all: } \pi = \frac{1/2}{1/2 + 1/2} = 0.5$$

sample, then check

share of careful among kept

nothing is learned

Notice: keeping = E-step (careful given right); retraining = M-step.